MONISH.S

192311132

CSE

CSA4312-INTERNET PROGRAMMING

CO1-AT1

1. Online Symposium Portal

1. Identify appropriate HTML5 semantic elements missing

The form is missing a proper HTML document structure and semantic elements such as:

- `<header>` – Displays the portal title.
- `<main>` – Contains the main registration content.
- `<section>` – Groups the symposium registration form.
- `<footer>` – Displays copyright or contact information.
- `<label>` – Provides accessible names for the input fields.

2. Modified standards-compliant HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Symposium Registration</title>
</head>
<body>
    <header>
        <h1>Online Symposium Portal</h1>
    </header>

    <main>
        <section>
            <h2>Student Registration</h2>
```

```
            <form action="/register" method="POST">
                <label for="name">Name:</label>
                <input type="text" id="name" name="name" required>

                <label for="email">Email:</label>
                <input type="email" id="email" name="email" required>

                <button type="submit">Submit</button>
            </form>
        </section>
    </main>

    <footer>
        <p>College Online Symposium</p>
    </footer>
</body>
</html>
```

3. Interpret the HTTP request

When the student submits the form, the browser sends an HTTP POST request to `/register`.

- Method: POST
- URL path: `/register`
- Data: Student name and email
- Content type: Usually `application/x-www-form-urlencoded` for this form
- Purpose: Sends registration data to the server for processing.

Example request body:

```
name=Monish&email=student%40example.com
```

4. Analyze the HTTP response cycle

1. The student enters the details and clicks Submit.
2. The browser validates the form fields.
3. The browser sends the POST request to the server.
4. The server receives and validates the submitted data.
5. The server processes the registration and returns an HTTP response.
6. The browser displays the resulting page or message.

Common response status codes:

- `200 OK` – Request processed successfully.
- `201 Created` – Registration resource created successfully.
- `400 Bad Request` – Invalid request data.
- `500 Internal Server Error` – Server-side failure.

5. Usability and accessibility improvements

- Use labels for every input field.
- Add `required` validation.
- Use suitable input types such as `email`.
- Provide clear success and error messages.
- Use keyboard-friendly controls and visible focus indicators.
- Use HTTPS to protect data in transit.
- Validate input on both the client and server.

2. Cross-Browser Compatibility Issue

1. Identify improper nesting and missing closing tags

Problems in the original code:

- Missing `<!DOCTYPE html>`.
- Missing `lang` attribute in the `<html>` element.
- Missing `<meta charset="UTF-8">`.
- Missing viewport metadata.
- The `<h1>` element is not closed.
- The `<p>` element is not closed.
- The `<body>` and `<html>` closing tags are missing.
- The heading and paragraph are not structured clearly.

2. How different browsers may interpret the structure

Browsers use HTML parsing rules to recover from many markup errors. They may automatically close certain elements or construct a document tree different from what the developer intended.

This can lead to differences in spacing, styling, element placement, and accessibility-tree interpretation. The exact behavior depends on the malformed markup and browser parser.

3. Missing semantic elements

- `<header>` – Website heading or introductory content.
- `<main>` – Primary page content.
- `<section>` – Groups related information.
- `<footer>` – Footer information.

4. Corrected W3C-compliant HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Company</title>
</head>
<body>
    <header>
        <h1>Welcome</h1>
    </header>

    <main>
        <section>
            <h2>About Us</h2>
            <p>Learn more about our company and services.</p>
        </section>
    </main>

    <footer>
        <p>Copyright 2026 Company</p>
    </footer>
</body>
</html>
```

5. Techniques for cross-browser compatibility

1. Use valid HTML5 and CSS.
2. Close tags correctly and nest elements properly.
3. Use a CSS reset or consistent base stylesheet where appropriate.
4. Use responsive design with media queries.
5. Avoid browser-specific features unless necessary.
6. Test the website in Chrome, Firefox, Edge, and Safari.
7. Use the W3C Markup Validation Service to check HTML errors.
8. Use developer tools to identify layout and CSS issues.

3. Form Submission Failure

1. Identify issues in the form configuration

The main problem is:

```
<button type="button">Login</button>
```

The `type="button"` attribute creates a normal button. It does not submit the form automatically.

Other concerns:

- The form uses GET for a password-based login.
- No labels or validation are provided.
- The server must validate credentials securely.

2. Why the HTTP request is not triggered

Clicking a button with `type="button"` does not trigger the form's default submission behavior. Unless JavaScript explicitly submits the form or performs a request, no form submission request is generated.

3. Role of the GET method

The GET method submits form data as URL query parameters.

Example:

```
/submit?username=Monish&password=example
```

This is inappropriate for passwords because URLs may be recorded in browser history, server logs, and other systems. GET is better suited to retrieving information, such as search results.

4. Corrected implementation

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login</title>
</head>
<body>
    <main>
        <h1>Student Login</h1>

        <form action="/submit" method="POST">
            <label for="username">Username:</label>
            <input type="text" id="username"
                   name="username" required>

            <label for="password">Password:</label>
            <input type="password" id="password"
                   name="password" required>

            <button type="submit">Login</button>
        </form>
    </main>
```

```
</body>
</html>
```

5. Improvements for secure data transmission

- Use HTTPS to encrypt data in transit.
- Use POST rather than GET for login credentials.
- Validate credentials on the server.
- Store passwords using a suitable salted password-hashing algorithm, never as plain text.
- Use secure session cookies with `HttpOnly`, `Secure`, and appropriate `SameSite` settings.
- Apply rate limiting to reduce brute-force login attempts.
- Never rely only on client-side validation for security.

Conclusion: Correct semantic HTML, valid nesting, appropriate form methods, and secure server-side handling improve accessibility, compatibility, and reliability across all three applications.